

PRACTICAL-8

Perform Statistical Parsing using PCFG in nltk

Statistical Parsing

Meaning:

Statistical (probabilistic) parsing uses probabilities to find the **most likely parse tree** for a sentence using a **Probabilistic Context-Free Grammar (PCFG)**.

Each grammar rule has a probability, and the total probability of a parse tree is the product of rule probabilities. Example:

```
S → NP VP [1.0]
NP → Det N [0.6] | Det N PP [0.4]
VP → V NP [0.5] | V NP PP [0.5]
```

The parser selects the tree with the **highest probability** for a sentence.

Parse Tree (Viterbi Parser) – Viterbi Parser is a statistical parser that finds the **most probable parse tree** using dynamic programming.

Code:

```
▶ # statistical parsing using pcfg in nltk
import nltk
from nltk import Nonterminal, PCFG
from nltk.parse import ViterbiParser

grammar = PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.6] | Det N PP [0.4]
VP -> V NP [0.5] | V NP PP [0.3] | V PP [0.2]
PP -> P NP [1.0]
Det -> 'a' [0.2] | 'the' [0.8]
N -> 'cat' [0.5] | 'dog' [0.5]
V -> 'chased' [0.69] | 'sat' [0.3] | 'ran' [0.01]
P -> 'with' [0.6] | 'in' [0.4]

""")

parser = ViterbiParser(grammar)

sentence = ['the', 'dog', 'chased', 'the', 'cat', 'with', 'a', 'dog']
print("Parsing sentence: ", ''.join(sentence))

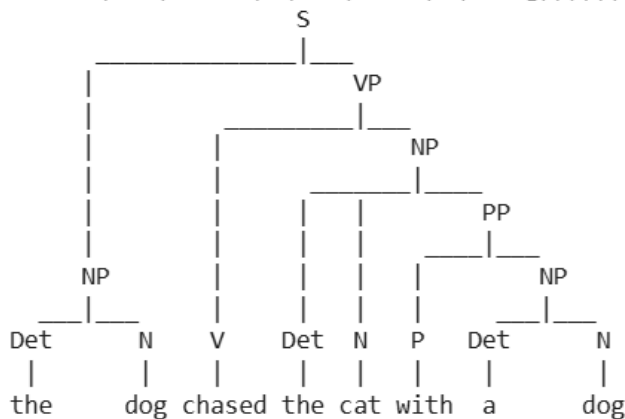
for tree in parser.parse(sentence):
    print("\nMost Probable Parse Tree:")
    print(tree)
    tree.pretty_print()
```

Output:

↔ Parsing sentence: thedogchasedthecatwithadog

Most Probable Parse Tree:

```
(S
  (NP (Det the) (N dog))
  (VP
    (V chased)
    (NP
      (Det the)
      (N cat)
      (PP (P with) (NP (Det a) (N dog)))))) (p=0.000476928)
```



InsideChart Parser

- Computes the total probability of all possible parses for a sentence (not just the best one).
- Used to estimate how likely a sentence is under a grammar.
- Useful in training and probabilistic evaluation.



Inside chart parser tree

```
import nltk
from nltk import Nonterminal, PCFG
from nltk.parse import ChartParser
from nltk.parse import InsideChartParser

grammar = PCFG.fromstring("""

S -> NP VP [1.0]
NP -> Det N [0.6] | Det N PP [0.4]
VP -> V NP [0.2] | V NP PP [0.6] | V PP [0.2]
PP -> P NP [1.0]
Det -> 'a' [0.2] | 'the' [0.8]
N -> 'cat' [0.5] | 'dog' [0.5]
V -> 'chased' [0.69] | 'sat' [0.3] | 'ran' [0.01]
P -> 'with' [0.3] | 'in' [0.4] | 'and' [0.3]
""")

parser = InsideChartParser(grammar)
sentence = ['the', 'dog', 'chased', 'a', 'dog', 'and', 'a', 'cat']

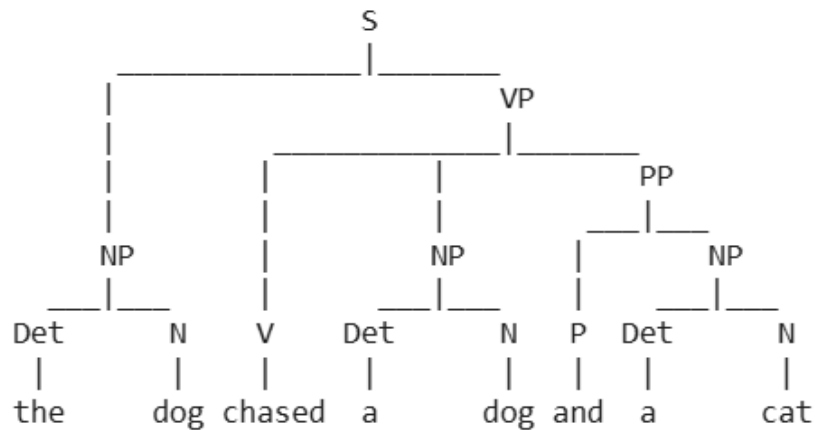
print("Parsing sentence: ", ''.join(sentence))
for tree in parser.parse(sentence):
    print("\nMost Probable Parse Tree:")
    print(tree)
    tree.pretty_print()
```

Output:

↔ Parsing sentence: thedogchasedadogandacat

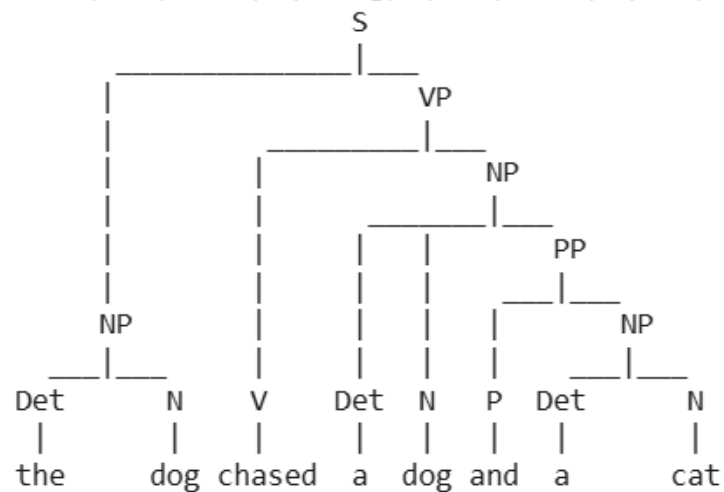
Most Probable Parse Tree:

```
(S
  (NP (Det the) (N dog))
  (VP
    (V chased)
    (NP (Det a) (N dog))
    (PP (P and) (NP (Det a) (N cat))))) (p=0.000107309)
```



Most Probable Parse Tree:

```
(S
  (NP (Det the) (N dog))
  (VP
    (V chased)
    (NP (Det a) (N dog) (PP (P and) (NP (Det a) (N cat))))) (p=2.38464e-05)
```



Feature Structures

Represent information as **attribute–value pairs**.

They allow **unification** (merging) if features match.

Unification Grammar

- Adds **linguistic features** like number, person, or tense to grammar rules.
- Ensures **agreement** (e.g., “He runs” vs “He run”).
- Uses **feature structures** to store and check these properties.

Code:

```

▶ import nltk
  from nltk import CFG
  from nltk.parse import ChartParser, FeatureChartParser
  from nltk.grammar import FeatureGrammar
  from nltk.sem import logic
  from nltk.sem.logic import Expression, Variable, ApplicationExpression

try:
    nltk.data.find('grammars/book_grammars')
except LookupError:
    nltk.download('book_grammars')

feature_grammar = FeatureGrammar.fromstring("""

    S -> NP[NUM=?n] VP[NUM=?n]

    NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n]
    NP[NUM=?n] -> PropN[NUM=?n]

    # VP must agree with subject, but can take any NP as object
    VP[NUM=?n] -> V[NUM=?n] NP[NUM=?x]
    VP[NUM=?n] -> V[NUM=?n]

    Det[NUM=sg] -> 'this' | 'a'
    Det[NUM=pl] -> 'these' | 'those'

    N[NUM=sg] -> 'dog' | 'cat' | 'book'
    N[NUM=pl] -> 'dogs' | 'cats' | 'books'

    V[NUM=sg] -> 'runs' | 'barks' | 'reads'
    V[NUM=pl] -> 'run' | 'bark' | 'read'

    PropN[NUM=sg] -> 'John' | 'Mary'
""")

feature_parser = FeatureChartParser(feature_grammar)
  
```

```

# Test sentences
test_sentences = [
    "this dog runs",      # Valid - singular agreement
    "these dogs run",     # Valid - plural agreement
    "this dog run",       # Invalid - number mismatch
    "these dogs runs"     # Invalid - number mismatch
]

print("Testing Subject-Verb Agreement:\n")
print("=" * 60)
for sent in test_sentences:
    tokens = sent.split()
    print(f"\nSentence: '{sent}'")
    trees = list(feature_parser.parse(tokens))
    if trees:
        print("✓ Valid - Agreement successful!")
        for tree in trees:
            print(tree)
            print()
    else:
        print("X Invalid - Unification failed (agreement error)")
print("=" * 60)

```

Output:

Testing Subject-Verb Agreement:

=====

Sentence: 'this dog runs'

✓ Valid - Agreement successful!

(S[]

(NP[NUM='sg'] (Det[NUM='sg'] this) (N[NUM='sg'] dog)))

(VP[NUM='sg'] (V[NUM='sg'] runs)))

Sentence: 'these dogs run'

✓ Valid - Agreement successful!

(S[]

(NP[NUM='pl'] (Det[NUM='pl'] these) (N[NUM='pl'] dogs)))

(VP[NUM='pl'] (V[NUM='pl'] run)))

Sentence: 'this dog run'

X Invalid - Unification failed (agreement error)

Sentence: 'these dogs runs'

X Invalid - Unification failed (agreement error)

=====

